

```
=====
1) What is Webservice ?
=====
```

=> Webservices are used to develop distributed applications.

=> If one application is communicating with another application then we will call it as distributed application.

Passport <-----> AADHAR App

MakeMyTrip <-----> IRCTC App

GPay <-----> Banking App

=> Every Distributed application should maintain intereoperability.

JAVA App <-----> Python App

Python App <-----> .Net App

.Net App <-----> Java App

Angular App <-----> JAVA / Python / DOT NET / Node JS

React App <-----> JAVA / Python / DOT NET / Node JS

=> Intereoperability means irrespective of the platform and language if applications are communicating then we will call them as Intereoperable applications.

```
=====
Q) Why we should develop Distributed application?
=====
```

-> Distributed applications are used for Business to Business Communication (B 2 B)

-> We can reuse the functionality of one project in another project.

Ex-1:

-> IRCTC project contains business logic to book train tickets.

-> MakeMyTrip application will communicate with IRCTC to book train tickets.

Note: In this scenario, MakeMyTrip application is re-using IRCTC application logic to book train tickets.

Ex-2 :

-> OPEN AI company developed GPT Model (LLM)

-> Other applications can use use GPT Model for AI implementation.

Distributed applications can be developed in 2 ways

1) SOAP Webservices (Outdated)

2) RESTFul Services (Trending)

=====
What is REST API ?
=====

** API stands for Application programming interface.

** REST stands for Representational State Transfer.

** REST API defines set of rules to establish business to business communication.

** Using REST API we can provide business services to other applications.

** REST API defines how to access one application from another application

- What is the URL Pattern
- What type of request
- Request Data
- Response Data

=====
REST API Architecture
=====

1) Provider

2) Consumer

=> Provider means the application which is providing business services to other applications.

=> Consumer means the application which is accessing business services from other applications.

Ex: IRCTC is a provider and MakeMyTrip & Yatra are consmers.

OPEN AI GPT Project is a provider, Ashok IT Web App is a consumer.

Note: We will use JSON to exchange data between provider & consumer.

=====
JSON
=====

=> JSON stands for Java Script Object Notation.

=> JSON represents data in key-value format.

Ex:

```
{  
    "id" : 101,  
    "name" : "Ashok",  
    "phone" : 8686868  
}
```

=> JSON is very light weight.

=> JSON is platform independent & language independent.

=> JSON is used to transfer data over a network.

=> Distributed applications will use JSON data for request & response.

Note: To work with json data in python, we have "json" module (in-built)

json.dumps() : Converts Python object into JSON string

json.dump() : Writes Python object into JSON file

json.loads() : Converts JSON string into Python object

json.load () : Reads JSON file and converts into Python object

```
-----  
import json  
  
student = {  
    "id": 101,  
    "name": "Ravi",  
    "course": "Python",  
    "fee": 15000  
}  
  
student_json = json.dumps(student, sort_keys=True, indent=4)  
print(student_json)  
print(type(student_json))  
  
print("-----")  
  
student = json.loads(student_json)  
print(student)  
print(type(student))  
  
print("-----")  
  
with open("student.json", "w") as file:  
    json.dump(student, file, indent=10)  
  
print("JSON file created successfully")  
  
print("-----")  
  
with open("student.json", "r") as file:  
    student = json.load(file)  
    print(student)  
-----
```

```
=====  
What is HTTP  
=====
```

=> HTTP stands for Hyper Text Transfer Protocol.

=> HTTP acts as mediator between Client & Server.

=> HTTP is stateless protocol
(can't remember conversation happened between client & server)

=> To develop REST APIs we need to know below concepts of HTTP

- 1) HTTP Request Structure
- 2) HTTP Response Structure
- 3) HTTP Methods
- 4) HTTP Status Codes

=====

HTTP Request Structure

=====

=> It contains below parts

- 1) Request Line (HTTP Method + Server URL)
- 2) Request Headers (Metadata) ==> (KEY - VALUE format)
- 3) Request Body (Payload ---> text / xml / json)

=====

HTTP Response Structure

=====

=> It contains below parts

- 1) Response Line (Status Code + Status MSG)
- 2) Response Headers (Metadata - K & V)
- 3) Response Body (Payload - text / xml / json)

=====

HTTP Methods

=====

- 1) GET
- 2) POST
- 3) PUT
- 4) PATCH
- 5) DELETE

=> GET method is used to get data from server (no request body)

=> POST method is used to send data to server (it creates new record at server)

=> PUT method is used to update the record (complete record update)

=> PATCH method is used for record partial update

=> DELETE method is used to delete record at server

=====
HTTP Status Codes
=====

2xx (200 to 299) : Success

4xx (400 to 499) : Client Error

5xx (500 to 599) : Server Error

=====
Develop REST API Using FAST API
=====

Step-1 : Create Python Project

Step-2 : Create "requirements.txt" file inside the project

Step-3 : Configure required libraries in 'requirements.txt' file

Step-4 : Create Virtual Environment and Activate it

Step-5 : Install the libraries in venv using 'pip'

```
$ pip install -r requirements.txt
```

Step-6 : create app.py file with "rest endpoint" methods

Step-7 : Run the application using uvicorn

```
Ex : uvicorn main:app --reload
```

Step-8 : Test the application using swagger documentation

```
URL : http://localhost:8000/docs
```

Note: When we hit above url, FastAPI automatically generates API documentation.

** With API documentation we can understand, What endpoints available, our API methods mapped to which type of HTTP requests, what are the request parameters, request data format, response data format.

** Using Swagger Documentation we can test the REST Endpoints also.

Note: We can use POSTMAN also for API testing (it won't give documentation, we need to provide api details to postman to send the request.)

```
-----  
  
from fastapi import FastAPI  
  
app = FastAPI()  
  
@app.get("/welcome")  
def get_welcome_msg():  
    return {"message" : "Welcome to FastAPI"}
```

```
@app.get("/greet")
def get_greet_msg():
    return {"message" : "Good Morning"}
```

```
=====
GET API Example
=====
```

=> GET API is used to fetch data from the server.

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/course")
def get_course():
    return {
        "course": "Gen AI with Python",
        "duration": "3 Months",
        "trainer": "Ashok"
    }
```

```
URL : http://127.0.0.1:8000/course
```

=> When we are using GET request we can send data to server in the URL...

=> We have 2 options to send data in the URL

1) Path Parameter

2) Query Parameter

```
=====
### What is Path Parameter
=====
```

```
-- Used to send data in URL directly
```

```
-- Need to represent its position in url template
```

```
from fastapi import FastAPI, HTTPException
from courses import courses
```

```
app = FastAPI()
```

```
@app.get("/courses")
def get_course():
    return courses
```

```
@app.get("/courses/{course_id}")
def get_course(course_id: int):

    course = courses.get(course_id)

    if course is None:
        raise HTTPException(
            status_code=404,
            detail="Course not found"
        )

    return course
```

```
=====
### What is Query Parameter
=====
```

=> Query parameter means passing value after ? in the URL.

Ex : `http://localhost:8000/courses?name=genai&trainer=ashok`

=> Query Parameters starts with ? and seperated by "&"

=> Query Parameters should present only at the end of the URL

```
@app.get("/course-search")
def search_course(search: str):

    result = []

    for course_id, course in courses.items():
        if search.lower() in course["course_name"].lower():
            result.append({
                "course_id": course_id,
                **course
            })

    return result
```

`http://localhost:8000/course-search?search=stack`

```
=====
POST API with Request Body
=====
```

=> POST API is used to send data to the server.

=> FastAPI uses Pydantic models to receive and validate request body data.

```
class Course(BaseModel):
    course_id: int
    course_name: str
    course_price: int

@app.post("/course", status_code=201)
def create_course(course: Course):
    # logic to insert data into db
    return {
        "message" : "Course created",
        "course" : course
    }
```

@@@ Assignment : Develop a REST API to perform CRUD operations using MYSQL Database.
